

1 Solution — GIAM 7.2

Problem 6

1.1 Problem

State necessary and sufficient conditions for the existence of an *Eulerian circuit* in a graph.

1.2 Definitions

A (multi-)graph $G = (V, E)$ has a vertex set V and an edge multiset E , where each edge connects two vertices (parallel edges and self-loops allowed; a self-loop at v contributes **2** to $\deg(v)$). An *Eulerian circuit* is a *closed walk* that traverses every edge of G exactly once. The graph is *connected* if there is a walk between any two vertices.

1.3 Theorem (Euler, 1736)

A finite (multi-)graph G with at least one edge has an Eulerian circuit if and only if both of the following hold:

- (C1) every vertex of G has *even* degree, and
- (C2) every edge of G lies in a single connected component
(equivalently, G is connected after deleting isolated vertices).

When G is itself connected (no isolated vertices to worry about), condition (C2) is automatic and the criterion collapses to:

A connected graph admits an Eulerian circuit iff every vertex has even degree.

We prove both directions. Necessity rests on the same parity / handshake observation behind Problem 2; sufficiency is the constructive *Hierholzer's algorithm*.

1.4 Proof of Necessity ($\Rightarrow$)

Suppose G has an Eulerian circuit C .

(C1) — every vertex has even degree. Fix any vertex v . Every time C visits v , it arrives along one edge and departs along a different edge (since C uses each edge only once, it cannot leave by the edge it arrived on). At the start/end of C — which is the same vertex v_0 by definition of “circuit” — the initial departure and final arrival also pair up. So the edges incident to v are partitioned into *pairs* (arrive, depart), forcing $\deg(v)$ to be even.

(C2) — all edges in one component. A walk cannot “jump” between connected components: every consecutive pair of edges in C shares a vertex, so the edges of C all belong to a single component. Since C uses *every* edge of G , all edges must lie in that one component. $\square$

1.5 Proof of Sufficiency ($\Leftarrow$) — Hierholzer’s Algorithm

Suppose (C1) and (C2) hold. We construct an Eulerian circuit by *Hierholzer’s algorithm* (1873). Without loss of generality discard isolated vertices: assume G is connected with every vertex of positive, even degree.

Step 1 — Greedy walk that always closes. Pick any vertex v_0 . Start a walk from v_0 , traversing any unused edge; at each subsequent vertex, follow another unused edge as long as one exists.

Claim. The walk *cannot get stuck* at any vertex $w \neq v_0$.

Proof of claim. Suppose the walk has just arrived at w via some edge. Count edge-uses at w : the walk has entered w some number of times and left some number of times, plus the final arrival. By the alternation “enter/leave/enter/leave/...”, the walk has used an *odd* number of edges at w so far. Since $\deg(w)$ is even (by C1), at least one edge at w remains unused — the walk can continue.

So the walk can only halt at v_0 itself. Since G has only finitely many edges, the walk halts in finitely many steps. Call this closed walk W_0 ; it uses some subset of the edges of G .

Step 2 — If W_0 covers E , we are done. Otherwise, let G' be the graph obtained by deleting the edges of W_0 from G . Two facts:

- G' still has all even degrees. At each vertex v , the walk W_0 used an even number of edges (two per visit, paired up — same argument as the necessity proof). So removing those edges from $\deg_G(v)$ — still even — leaves $\deg_{G'}(v)$ even.
- There is a vertex of W_0 with a G' -edge. If not, then every vertex visited by W_0 has all of its G -edges accounted for in W_0 , so the edges of W_0 form their own connected component disjoint from the rest — contradicting (C2).

Pick such a vertex u on W_0 .

Step 3 — Splice in a sub-circuit at u . Apply Step 1 inside G' , starting at u . This produces a closed walk W_1 from u back to u , using only edges of G' (i.e., not in W_0).

Splice W_1 into W_0 at the vertex u : when W_0 reaches u , detour through W_1 (returning to u), then continue W_0 . The resulting closed walk uses every edge of $W_0 \cup W_1$ exactly once — i.e., *strictly more* edges than W_0 .

Step 4 — Iterate. Replace W_0 by the spliced walk and return to Step 2. Each iteration uses strictly more edges; since $|E|$ is finite, the process terminates with all edges of G used. The final walk is an Eulerian circuit. ■

1.6 Examples

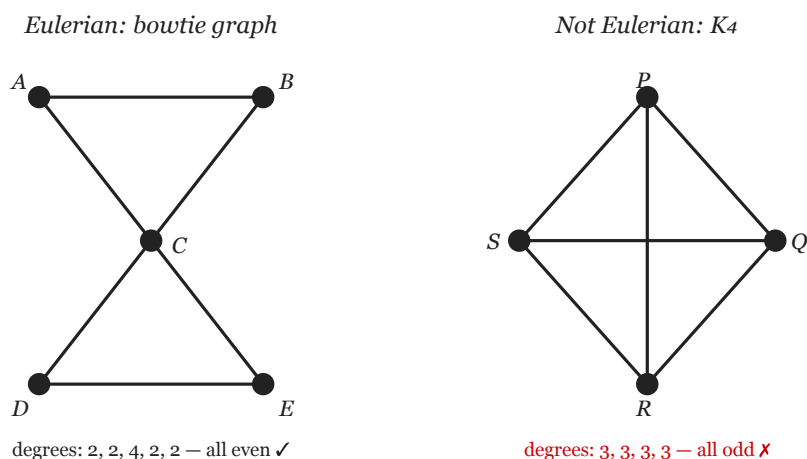


Figure 1.1: Left: the ‘bowtie’ graph (two triangles sharing a vertex) has degree sequence (2, 2, 4, 2, 2) — all even — so it has an Eulerian circuit. Right: K_4

has degree sequence $(3, 3, 3, 3)$ — all odd, four of them — so it has neither an Eulerian circuit nor an Eulerian path.

- **Bowtie (Eulerian).** Two triangles sharing vertex C . Degrees are $A = 2, B = 2, C = 4, D = 2, E = 2$ — all even, graph connected. An explicit Eulerian circuit:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow C \rightarrow A.$$

Six edges, each used once, returning to the start. ✓

- K_4 (**not Eulerian**). Every vertex has degree **3**; there are four odd-degree vertices, so (C1) fails. By the theorem, no Eulerian circuit. (And no Eulerian path either — that would need exactly two odd-degree vertices.)

1.7 Eulerian Path (Companion Result)

An *Eulerian path* is an *open* walk (distinct endpoints) traversing every edge once. The companion theorem:

A connected graph G has an Eulerian path from u to v (with $u \neq v$) iff exactly $\{u, v\}$ are the odd-degree vertices.

Proof sketch. Necessity is as before: interior vertices of the path are visited in pairs (enter, leave); the two endpoints contribute one extra unpaired incidence each. Sufficiency reduces to the circuit case: add a phantom edge uv to G , making u and v even-degree; the result has all even degrees and is connected, so by Euler's theorem it has an Eulerian circuit. Deleting the phantom edge from the circuit gives an Eulerian path from u to v in G .

This is exactly the criterion applied to the four graphs of Problem 1 of this section: count the odd-degree vertices — **0** for a circuit, **2** for a path, anything else means neither.

1.8 Remarks

- **Königsberg.** Euler's original **1736** paper introduced the modern graph-theoretic formulation specifically to solve the Königsberg bridge problem. That graph has **4** vert-

ices (banks/islands) with degrees **3, 3, 3, 5** — all odd. By necessity, no Eulerian circuit and no Eulerian path: it is impossible to walk the city crossing every bridge once.

- **Hierholzer’s algorithm.** The constructive proof above is by Carl Hierholzer (1873). Implemented with appropriate data structures (a doubly-linked walk supporting $O(1)$ splicing, and a queue of “splice candidate” vertices), it runs in $O(|E|)$ time — linear in the edge count.
- **Directed graphs.** A digraph admits an Eulerian directed circuit iff its underlying undirected graph is connected *and* every vertex has $\deg^+(v) = \deg^-(v)$ (in-degree equals out-degree). The proof is the directed analog of the above.
- **Chinese Postman Problem.** When (C1) fails, one cannot traverse every edge exactly once and return to start. The *Chinese postman* (or *route inspection*) problem asks for the shortest closed walk that traverses every edge *at least* once. It is solved by a clever reduction to minimum-weight matching of the odd-degree vertices — itself solvable in polynomial time — turning a parity obstruction into an algorithmic question.
- **Connection to Problem 2.** Necessity of (C1) sits on the same parity reasoning as Problem 2’s “every graph has an even number of odd nodes.” Both are corollaries of the basic identity $\sum_v \deg(v) = 2|E|$.